

Linux Server Operations Lab

基本点検 + 応用第1～8弾 | 実機検証・障害対応 総まとめ

更新日：2026年10月11日

対象：VirtualBox / Ubuntu 24.04 / tokyoサーバ中心

実施済み：基本点検、nginx停止、nginx設定ミス、403権限エラー、ディスク容量不足、DNS・通信、SSH、バックアップ・復元、最終第8弾の設定ミス検知と復旧。

この更新版の読み方

次の13ページは第7弾までの既存の記録です。その後に第8弾の実施記録と総括を追加しています。既存ページにある「第8弾未実施」という記述は、更新前の記録時点を示すものです。現在は第8弾の検証まで実施済みです。

記録上の重要な区別

第8弾で実際に再現したのは「nginx設定ファイルの文法ミス」です。複数の障害を同時発生させる演習や、nginxを停止させたうえでの復旧は行っていません。今回の成果は、稼働中のサービスへの影響を広げず、事前確認・原因特定・バックアップ復元・動作確認を行えたことです。

GitHub公開前の確認

サーバ名、IPアドレス、ログ、公開可能な情報かを確認してください。操作を実施した事実と、一般的な説明・未検証の推測を区別して記載しています。

Linux Server Operations Lab

実機検証・障害再現・切り分け・復旧の総まとめ | 基本点検 + 応用第1～7弾

作成日：2026年10月11日 / 対象：VirtualBox 上の Ubuntu 24.04 学習環境

この資料は、これまでの会話で共有された実行コマンド・端末出力・学習記録を基に構成しています。記録にない数値や成功結果は補っていません。手順を再実行する際は、現在のサーバ状態を必ず確認してください。

最初に：この資料の読み方

単なるコマンド集ではなく、「何を調べたかったか」「何が起きたか」「なぜその操作を選んだか」を結び付けて復習するための資料です。実機で操作できたことと、理由を自分で説明できることは分けて考えます。

区分	扱う内容
基本点検	ホスト・負荷・CPU/メモリ/ディスク・サービス・HTTP・ネットワーク・ユーザー/権限・ログ・更新
応用 第1～3弾	nginx停止、設定ミス、403権限エラー
応用 第4～6弾	ディスク満杯、DNS/通信、SSH接続・認証
応用 第7弾	cp / tar / tar.gz / rsyncによるバックアップ・復元
未実施	第8弾：複合障害の切り分け・復旧

検証環境

ホスト	IPアドレス	主な役割
controller	192.168.56.10	管理・SSH接続元
tokyo	192.168.56.11	今回の主な検証対象
yokohama	192.168.56.12	別の管理対象
omiya	192.168.56.13	別の管理対象

tokyo には VirtualBox の NAT 側（例：10.0.2.15、ゲートウェイ 10.0.2.2）とホストオンリー側（192.168.56.11）がある。SSH、nginx、ログ、ファイル操作などは主に tokyo 上で検証。

検証の共通フロー

1. 正常状態を確認 → 2. 変更・障害を再現 → 3. 症状を観察 → 4. 原因候補を絞る → 5. 必要最小限の修正 → 6. 復旧確認 → 7. 記録・再発防止を考える。

第0部：Linuxサーバ基本点検（実施済み）

目的：Linuxサーバを見たとき、「正常そうかどうか」を根拠を持って判断するための基本確認。2026年10月初旬までに一通り実施し、基礎編の振り返りPDFも作成済み。

確認対象	主なコマンド例	何を知りたいか
操作対象	hostname / pwd / whoami	どのサーバで、誰として、どこにいるか
稼働・負荷	uptime	稼働時間、ロードアベレージ
CPU/メモリ	free -h など	メモリの余裕や利用状況
ディスク	df -h / du -h	ファイルシステムの使用率と容量消費
サービス	systemctl status nginx	nginxが起動しているか
HTTP	curl -I http://localhost	HTTP応答が返るか
通信/DNS	ip addr / ip route / ping / dig	アドレス、経路、疎通、名前解決
ログ	journalctl / tail	いつ、どのような出来事があったか
権限・更新	ls -l / id / sudo / apt	ユーザー権限とパッケージの状態

基本点検で理解したいこと

「サービスがactive」だけでWebページが正常とは限らない。systemctlはサービス、ssは待受、curlはHTTP応答、journalctlは履歴を確認する。それぞれ観察している層が違う。更新や再起動は影響を考えたうえでいい、再起動後にサービスが戻ったかも確認する。

第1弾：nginx停止 → 障害検知 → 復旧

目的：Webサイトに接続できない場合、サービス停止とポート待受・HTTP失敗の関係を実機で確認する。

```
sudo systemctl stop nginx
systemctl is-active nginx
curl -I http://localhost
sudo ss -lntp | grep :80
sudo journalctl -u nginx --no-pager
sudo systemctl start nginx
curl -I http://localhost
```

観察した事実：nginx停止後はサービスが inactive となり、HTTP接続に失敗。80番ポートの待受がなくなり、ログからサービス停止の経緯を調べた。起動後は HTTP 200 を確認した。

原因：nginxサービスを意図的に停止したため。復旧：サービスを起動し、状態だけでなくHTTP応答まで確認。
再発防止の視点：サービス監視・死活監視、再起動時の影響確認。

第2弾：nginx設定ミス → 起動失敗 → 修正

目的：設定ファイルの構文エラーが、nginxの起動にどう影響するかを調べる。

```
sudo nginx -t  
  
sudo systemctl stop nginx  
  
sudo systemctl start nginx  
  
sudo journalctl -u nginx --no-pager  
  
sudo nginx -t  
  
sudo systemctl start nginx
```

再現したミス：nginx設定の `worker_processes auto;` の末尾セミコロンを欠落させた。nginx -t
で構文エラーを確認。既に起動していたnginxは、その時点では HTTP 200
を返す場面があったが、停止後の再起動には失敗した。

復旧：セミコロンを戻す → nginx -t で構文正常を確認 → 起動 → systemctl / ss / curl で復旧確認。学び：設定
ファイルが壊れていても稼働中のプロセスが直ちに止まるとは限らない。設定変更後は再起動前に構文確認する。

第3弾：nginx 403 Forbidden とファイル権限

目的：HTTP 403 が返ったとき、Webサーバ停止ではなくアクセス権限が原因になることを確認する。

```
sudo chmod 000 /var/www/html/permission-test.html
curl -I http://localhost/permission-test.html
sudo tail -n 30 /var/log/nginx/error.log
sudo chmod 644 /var/www/html/permission-test.html
curl -I http://localhost/permission-test.html
```

観察した事実：テストファイルは初め HTTP 200。権限を 000 にすると HTTP 403。nginx error.log に open() ... failed (13: Permission denied) が記録された。権限を 644 に戻すと HTTP 200 に復旧。トップページも HTTP 200 を確認。

原因：Webサーバが対象ファイルを読み取れない権限になったこと。重要：403は「Webサーバに到達しているがアクセスを許可できない」場合の応答。ファイル権限以外にも設定等が原因となることはある。

第4弾：ディスク容量不足（隔離したテスト領域）

目的：本番相当のルート領域を満杯にせず、安全な小容量ファイルシステムで No space left on device を再現する。

```
df -h
du -h /mnt/disk-lab
dd if=/dev/zero of=/mnt/disk-lab/test-data.bin bs=1M count=50
dd if=/dev/zero of=/mnt/disk-lab/overflow.bin bs=1M count=40
df -h /mnt/disk-lab
rm /mnt/disk-lab/overflow.bin
df -h /mnt/disk-lab
```

準備：/var/tmp/disk-lab.img（100MBの実験用イメージ）を ext4 で初期化し、ループデバイスとして /mnt/disk-lab にマウント。初期の利用可能容量は約83MB。

観察した事実：50MiBのデータ作成後、追加40MiBの書き込み途中で No space left on device が発生。dfで利用率100%、duで主な使用ファイルを確認。overflow.bin削除後は利用率約61%に戻り、テスト書き込みが成功した。

終了処理：テスト用ファイルシステムをアンマウントし、実験用イメージとディレクトリを削除して片付けた。学び：dfはファイルシステムの残量、duはディレクトリやファイルの容量消費を調べる。容量不足を調べるときは無闇にファイルを消さず、対象と影響を確認する。

第5弾：ネットワーク・DNS障害の切り分け

目的：IP通信ができることと、ドメイン名をIPアドレスに変換できることは別だと理解する。

```
ip addr
ip route
ping -c 3 10.0.2.2
ping -c 3 1.1.1.1
getent ahostsv4 example.com
resolvectl status
dig example.com A
dig @192.0.2.1 example.com A +time=2 +tries=1
ip route get 192.0.2.1
curl -I --max-time 10 https://example.com
```

観察した事実：NAT側からゲートウェイ 10.0.2.2 と 1.1.1.1 への疎通を確認。通常の dig example.com A は NOERROR でアドレスを返した。一方、文書用IP 192.0.2.1

をDNSサーバとして指定した問い合わせはタイムアウトした。通常の問い合わせと HTTPS アクセスは成功。

```
curl -I --max-time 10 --resolve example.com:443:104.20.23.154 https://example.com
```

追加検証：curl の --resolve を用いて接続先IPを指定し、HTTPSの応答を確認。詳細ログでは対象IPへの接続、TLS証明書検証、HTTP/2応答を確認した。

注意：192.0.2.1 はテスト用の予約アドレスであり、実在のDNSサーバを停止させたわけではない。「DNS問い合わせがタイムアウトする状況」を模擬した。DNS障害と断定する前に、IP疎通、名前解決、HTTP/TLSを切り分ける。

第6弾：SSH接続・認証障害

目的：SSHのサービス稼働、ポート待受、TCP接続、認証、ログの違いを実機で確認する。

```
systemctl status ssh --no-pager
sudo ss -lntp | grep ':22'
ssh -p 2222 -o ConnectTimeout=5 localhost
sudo ss -lntp "( sport = :2222 )"
ssh -p 22 -o ConnectTimeout=5 localhost hostname
sudo journalctl -t sshd --since "10 minutes ago" --no-pager
sudo sshd -T | grep -E '^(port|pubkeyauthentication|passwordauthentication|permitrootlogin) '
```

観察した事実：ssh.service は active (running)。ss で 0.0.0.0:22 と [::]:22 の待受を確認。localhost の2222番ポートは Connection refused

で、ssにも2222番の待受なし。22番への接続はパスワード認証に成功し、hostname の結果は tokyo。

認証実験：クライアント側で公開鍵・パスワード認証を使わない指定をして接続すると Permission denied (publickey,password)。ログには Accepted password と、認証前に接続が閉じられたことを示す [preauth] が見られた。これはパスワード誤入力の証拠とは限らない。

有効設定の例：port 22、pubkeyauthentication yes、passwordauthentication yes、permitrootlogin without-password。sshd -T

は設定評価結果を表示するが、Match条件や稼働中プロセスの状態まで完全に代替するものではない。

表示	主に確認する箇所
Connection refused	指定ポートの待受、サービス、拒否設定
Connection timed out	経路、ファイアウォール、応答の有無
Permission denied	ユーザー、公開鍵、認証設定、認証ログ

補足：Connection timed out のケースは復習用の架空シナリオであり、SSH実機で再現した結果ではない。

第7弾：バックアップと復元（実行記録）

目的：ファイルを正常な状態に戻すための基本操作を体験し、コピー・アーカイブ・圧縮・同期の違いを理解する。
作業場所：/home/tokyo/backup-restore-practice。

A. 事前確認と安全な実験場所

```
pwd
mkdir backup-lab
ls -ld backup-lab
ls -la backup-lab
mkdir backup-restore-practice
cd backup-restore-practice
```

既存の backup-lab を作ろうとした際に File exists が発生。ls -ld / ls -la で確認すると、data、full_backup.tar、full_backup.tar.gz、restore、restore_gz、rsync_backup など既存データが存在した。削除せず、新しい backup-restore-practice を作成した。

学び：エラーが出てもすぐ削除・上書きせず、対象の正体と中身を確認する。

B. cpでバックアップし、誤変更から復元

```
nano server.conf
cat server.conf
cp server.conf server.conf.bak
ls -l
diff server.conf server.conf.bak
nano server.conf
diff server.conf server.conf.bak
cp server.conf.bak server.conf
cat server.conf
```

最初の内容：server_name=tokyo / service=nginx / status=normal。バックアップ作成後に元ファイルだけ status=error に変更。diffは「3c3」「< status=error」「> status=normal」を表示した。バックアップから上書きコピーして status=normal へ復元できた。

重要：cp コピー元 コピー先。バックアップ時と復元時ではコピー方向が逆になる。拡張子 .bak は識別用の名前であり、特別な保護機能ではない。

C. tarで3ファイルを1つにまとめ、復元

```
nano network.conf
nano service.conf
tar -cvf config_backup.tar server.conf network.conf service.conf
tar -tvf config_backup.tar
mkdir restore-test
tar -xvf config_backup.tar -C restore-test
```

```
diff server.conf restore-test/server.conf
```

確認した内容：server.conf（46バイト）、network.conf（53バイト）、service.conf（36バイト）。tar -tvfでアーカイブ内の3ファイルを確認。restore-testへ展開し、差分比較で一致を確認。

```
diff -r --exclude='*.bak' --exclude='*.tar' --exclude='restore-test' . restore-test
```

除外条件を付けて元ディレクトリと復元先を比較し、差分表示なし。tarの -c は作成、-t は一覧、-x は展開、-f はアーカイブ名、-C は処理先ディレクトリの指定。小文字cと大文字Cは別の意味。

D. tar.gzで圧縮・展開

```
tar -czvf config_backup.tar.gz server.conf network.conf service.conf
```

```
ls -lh config_backup.tar config_backup.tar.gz
```

```
mkdir restore-gzip-test
```

```
tar -xzf config_backup.tar.gz -C restore-gzip-test
```

```
diff server.conf restore-gzip-test/server.conf
```

```
diff -r restore-test restore-gzip-test
```

観察した事実：通常の config_backup.tar は10K、gzip圧縮した config_backup.tar.gz は251バイト。3ファイルを展開し、通常tarからの復元先と比較して差分なし。圧縮効果はデータ内容に依存し、常にこの割合で小さくなるわけではない。

第7弾：rsyncの検証（続き）

E. rsyncの初回コピー、再実行、変更後の同期

```
rsync --version  
  
mkdir rsync-test  
  
rsync -avn server.conf network.conf service.conf rsync-test/  
rsync -av server.conf network.conf service.conf rsync-test/  
rsync -av server.conf network.conf service.conf rsync-test/  
  
nano network.conf  
  
rsync -av server.conf network.conf service.conf rsync-test/  
  
cat rsync-test/network.conf
```

rsync version 3.2.7を確認。-avn（dry-run）では network.conf / server.conf / service.conf が対象と表示され、実コピーはなし。-av の初回実行で3ファイルが表示された。変更なしの2回目は個別ファイル名が表示されず、転送不要と判断された。

その後、元の network.conf を status=normal から status=updated に変更して再同期すると、network.conf のみ表示。コピー先の cat でも status=updated を確認した。

バックアップと同期は違う

手法	得意なこと	注意点
cp	単一・少数ファイルの簡単な複製	同名上書き、同一ディスク故障には弱い
tar	複数ファイルを1つにまとめる	まとめるだけでは圧縮されない
tar.gz	まとめてgzip圧縮する	圧縮率はファイルによって異なる
rsync	差分を見ながら効率的に同期	誤変更もコピー先へ反映され得る

今回のrsyncは「現時点の状態を同期する」実験。過去の正常な状態へ戻せる世代バックアップ、別ディスク・別ホストへの保存、復元テストは今後の実運用設計で検討する。

実験時に起きた小さなトラブルと判断

事象	原因・切り分け	対応
mkdir: File exists	同名の既存ディレクトリがあった	lsで調査し、既存データを残して新規作業場所を作成
SSHの2222番で拒否	指定ポートに待受がなかった	ssで待受を確認し、22番との違いを理解
diffが何も表示しない	比較した内容に差がない	復元内容の一致確認として活用
rsync再実行でファイル名なし	更新対象がなかった	1ファイルだけ変更して再同期し挙動を比較

横断復習：症状から調査手段を選ぶ

症状・目的	最初に見ること	確認の例
Webが表示されない	nginxサービス・待受・HTTP	systemctl / ss / curl
HTTP 403	アクセス権限とエラーログ	ls -l / nginx error.log
nginx起動失敗	設定の構文・サービスログ	nginx -t / journalctl
容量不足	ファイルシステム残量と使用箇所	df -h / du -h
名前解決が怪しい	IP疎通とDNS応答を分離	ping / dig / resolvectl
SSH拒否	サービス・22番待受	systemctl / ss
SSH認証失敗	ユーザー・鍵・認証ログ	journalctl -t sshd
復元できたか	元データと復元データの一致	cat / diff

実務を想定した確認チェックリスト

- ☐ 作業前：接続先ホスト・現在地・権限・変更対象を確認したか
- ☐ 正常時の状態：HTTP応答・サービス状態・設定などを記録したか
- ☐ 障害再現：変更箇所と影響範囲を説明できるか
- ☐ 切り分け：症状に合う確認コマンドを選んだか
- ☐ 復旧：修正後にサービス・通信・ファイル内容を確認したか
- ☐ バックアップ：コピー先が正しいか、元のファイルを上書きしないか
- ☐ 復元：展開・コピーだけでなく、diffや実際の動作で確認したか
- ☐ 片付け：テスト用設定・ファイル・マウントを正常に戻したか
- ☐ 記録：実行コマンド・実際の出力・原因・修正・学びを分けて残したか

次に取り組む第8弾（未実施）

複合障害の切り分け・復旧。1つの原因を決め打ちせず、サービス、ポート、設定、権限、ディスク、DNS、SSHなどの観点を組み合わせて、症状 → 仮説 → 検証 → 修正 → 復旧確認を一貫して行う予定。現時点では実機結果なし。

GitHub・note用の記録テンプレート

タイトル：何が起きたか（例：nginxは動いているのに403になる理由を調べた）

背景：なぜこの実験を行ったか／正常状態：事前に確認したこと／再現手順：どこをどう変えたか／症状：コマンドと実際の出力／切り分け：原因候補と確認順／原因：ログ・結果に基づく説明／復旧：修正内容と検証結果／再

発防止：次に備える方法／未解決：実験では確かめていない点。

記事では「実機で確認した事実」と「一般的な説明・仮説」を分け、実務経験として誇張しない。スクリーンショットは機密情報・ホスト名・IPアドレス等を公開前に確認する。

資料の範囲と注意

この総まとめは、Linux Server Operations Lab の基本点検と応用第1～7弾を対象とする。RestaurantOps Automation Lab（Ansible 3店舗管理）や Excel×Python×Ansible の詳細なPhase記録は別プロジェクトであり、ここでは統合していない。個々のコマンド出力の全文や全スクリーンショットは含まず、会話中で確認できた重要な結果を要約した。

以上。次の目標は「コマンドを打てる」だけでなく、「症状に応じて確認手段を選び、その理由を説明できる」状態を目指すこと。

第8弾：設定ミスの発見と安全な復旧

目的・シナリオ

tokyoサーバのnginxが正常稼働している状態から、設定ファイルに意図的な文法ミスを加える。設定テストで異常を検出し、稼働中のHTTP応答を確認したうえで、事前バックアップから設定を戻す。

1. 障害前の正常状態

```
systemctl status nginx --no-pager
```

確認結果：Active: active (running)。起動時のExecStartPre設定テストは status=0/SUCCESS。

```
curl -I http://localhost
```

確認結果：HTTP/1.1 200

OK。これはlocalhostに対するHTTP応答であり、外部からのアクセスまでは検証していない。

2. バックアップと同一性の確認

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.phase8.bak
```

```
sudo diff /etc/nginx/nginx.conf /etc/nginx/nginx.conf.phase8.bak
```

確認結果：diffで差分表示なし。設定変更前の内容をバックアップとして保持。

3. 意図的な設定ミス

```
sudo nano /etc/nginx/nginx.conf
```

worker_processes auto; の末尾セミコロンを削除して worker_processes auto とし、保存した。

```
sudo nginx -t
```

実測結果：invalid number of arguments in "worker_processes" directive in /etc/nginx/nginx.conf:3。configuration file test failed。

4. 設定に異常があってもHTTPは正常

```
curl -I http://localhost
```

実測結果：HTTP/1.1 200

OK。設定ファイルの変更をまだ読み込ませていないため、既存のnginxプロセスは正常な設定で応答を続けた。

5. バックアップから復旧

```
sudo cp /etc/nginx/nginx.conf.phase8.bak /etc/nginx/nginx.conf
```

```
sudo nginx -t
```

実測結果：syntax is ok / test is successful。設定の文法チェックが成功。

```
curl -I http://localhost
```

実測結果：HTTP/1.1 200 OK。復旧後もHTTP応答を確認。

結果・学習ポイント

「設定ファイルが壊れている」と「現在稼働中のサービスが停止している」ことは異なる。設定変更後は、サービス再起動前に `nginx -t` を行い、問題があれば変更を戻すことで影響を抑えられる。

今回実施していないこと

nginxの停止・再起動、同時に複数の障害を発生させた切り分け、外部クライアントからの通信検証、復旧後の全機能試験は未実施。これらを成功済みとして扱わない。

全8テーマの振り返り

第1弾：nginx停止と復旧 ／ 第2弾：nginx設定文法ミス ／ 第3弾：403 Forbiddenと権限 ／

第4弾：隔離環境での容量不足 ／ 第5弾：DNS・通信 ／ 第6弾：SSHの待受・認証 ／

第7弾：`cp`・`tar`・`tar.gz`・`rsync` ／ 第8弾：正常性確認から設定ミス検知、バックアップ復元、HTTP再確認。

次の課題

各コマンドについて「何を確認するために使うか」を説明できるよう復習する。今回の第8弾は複合障害そのものではなく、障害対応の一連の判断をつなげる演習であった。